

Outpatient Clinic Management Database

A read-heavy database for the busiest part of a hospital

CEN 202 · Database Management Systems · Epoka University

Renis Kraishta · Kostika Koca · Joen Ahmeti · Leandro Karafili · Endi Kambo

The Problem

Hospitals run on data, but a typical outpatient clinic juggles patients, doctors, prescriptions, and bills in spreadsheets and on paper.



Lost track of patients

Phone numbers in one file, history in another, no single source of truth.



Prescription errors

No structured way to record which medications go with which prescription.



Billing chaos

Hard to know who has paid, who hasn't, and what insurance should cover.

A relational database solves all three.

Our Scope

Outpatient care: same-day visits, no hospital admissions

Patient	**Doctor**	**Department**
Appointment	**Diagnosis**	**Prescription**
Medication	**Bill**	**Insurance**

By the numbers

9 entities

10 tables

9 relationships

25 queries

5 indexes

A single, well-defined slice of the hospital done thoroughly, not a sprawl done sloppily.

Two Pillars of Our Design

A good database is both correct and fast. We chose techniques for each.



Normalization

Correctness

- Every fact in exactly one place
- No duplicate phone numbers
- No way to corrupt the data
- Decomposed to Third Normal Form (3NF)



Indexing

Performance

- Lightning-fast lookups (B-tree, O(log n))
- Schedule lookups by date
- Patient search by surname
- Unpaid-bill reports in milliseconds

Two halves of the same coin: normalized data stays clean, indexed data stays quick.

Why Normalize?

Imagine cramming everything into one big table…

AppID	Patient	Phone	Doctor	Department	Medication
A101	Duart Lani	555-0101	Dr. Clark	Cardiology	Atenolol
A101	Duart Lani	555-0101	Dr. Clark	Cardiology	Aspirin
A102	Liam Brown	555-0104	Dr. Hall	Orthopedics	Ibuprofen
A103	Duart Lani	555-0101	Dr. Clark	Cardiology	Lisinopril



The fix: split into 10 tables, each with a single subject.

- Each patient lives in exactly one row of Patient
- Each department lives in exactly one row of Department
- Appointments reference patients and doctors by ID
- **All three anomalies become impossible by construction**

1 **Insertion anomaly**

Can't add a new doctor without inventing a fake appointment.

2 **Update anomaly**

Change Lani's phone? Update it 3 times — or it disagrees with itself.

3 **Deletion anomaly**

Delete the last appointment for Cardiology? Lose the department info.

The Wins from Normalization



No data duplication

Every fact lives in exactly one place.



Integrity guaranteed

The schema itself prevents inconsistent data.



Easier maintenance

Updating a phone number? One row, not many.



Clearer structure

Each table has one purpose. Easy to extend.

Cost: more JOINs at query time → which leads us to indexing.

The Idea of Indexing

Without an index, the database reads every row. With one, it can jump straight to the answer.

Without index

Read every row, one by one.

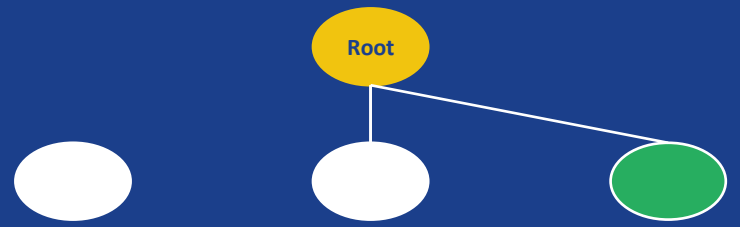


→ scan all 10 to find row 8

O(n)

With B-tree index

Jump down a balanced tree, level by level.



→ 2 levels to find row 8

O(log n)

On a million rows: ~20 comparisons instead of a million. That's the gap that makes a clinic feel responsive.

Indexes in Our Project

Five indexes: one for each thing a clinic does dozens of times a day.

Front desk	*Pulling today's schedule.*	`Appointment(AppointmentDate)`
Reception	*Looking up a walk-in patient.*	`Patient(LastName)`
Doctors	*Pulling a patient's history.*	`Prescription(AppointmentID)`
Billing	*Generating unpaid-bill reports.*	`Bill(PaymentStatus)`
Management	*Doctor workload analytics.*	`Appointment(DoctorID)`

Trade-off: tiny write overhead in exchange for dramatically faster reads. A clinic reads far more than it writes so we win.

Indexes Firing in Real Code

Query 8 — "Show me everyone scheduled for March 4th." Two indexes fire at once.

```
SELECT a.AppointmentID, a.AppointmentTime,
       doc.FirstName, doc.LastName,
       p.FirstName, p.LastName
FROM Appointment a, Doctor doc, Patient p
WHERE a.DoctorID = doc.DoctorID
  AND a.PatientID = p.PatientID
  AND a.AppointmentDate = '2026-03-04';
```



idx_appointment_date

Fires on the date filter.

Database jumps straight to the 2 appointments on March 4th — skips the other 18.



idx_appointment_doctor

Fires on the join condition.

Engine finds matching doctor rows in O(log n) instead of scanning the table.

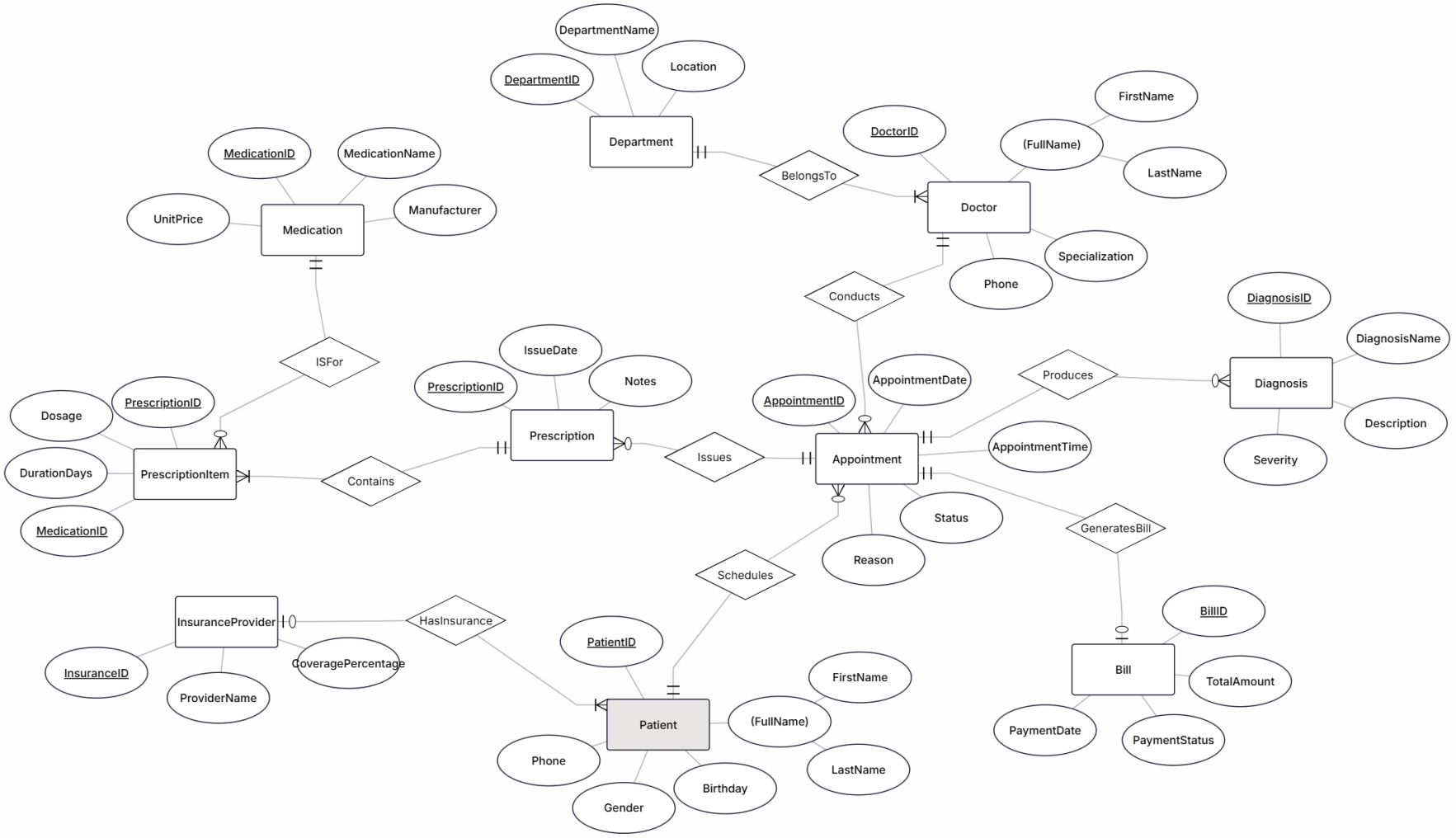
Without indexes vs With indexes

Without: scan all 20 appointments to find matching dates, then scan all 7 doctors per row to match the doctor.

With: B-tree jumps to March 4th in 2 levels. Joins use index-backed lookups. ***Same SQL, different execution.***

The ER Diagram

All 9 entities + the PrescriptionItem intersection — full cardinalities and participation.



Relationships in Plain English

Each relationship and how its two sides must behave.

1 BelongsTo

Department – Doctor

A doctor must belong to exactly one department; a department has one or many doctors.

2 Conducts

Doctor – Appointment

An appointment has exactly one doctor; a doctor can conduct zero or many appointments.

3 Schedules

Patient – Appointment

An appointment has exactly one patient; a patient can schedule zero or many appointments.

4 HasInsurance

Patient – InsuranceProvider

A patient may have zero or one insurance provider; a provider covers one or many patients.

5 Produces

Appointment – Diagnosis

A diagnosis comes from exactly one appointment; an appointment may produce zero or many diagnoses.

6 Issues

Appointment – Prescription

A prescription is issued at exactly one appointment; an appointment may issue zero or many prescriptions.

7 GeneratesBill

Appointment – Bill

A bill belongs to exactly one appointment; an appointment generates zero or one bill.

8 Contains

Prescription – PrescriptionItem

An item belongs to exactly one prescription; a prescription must contain one or many items.

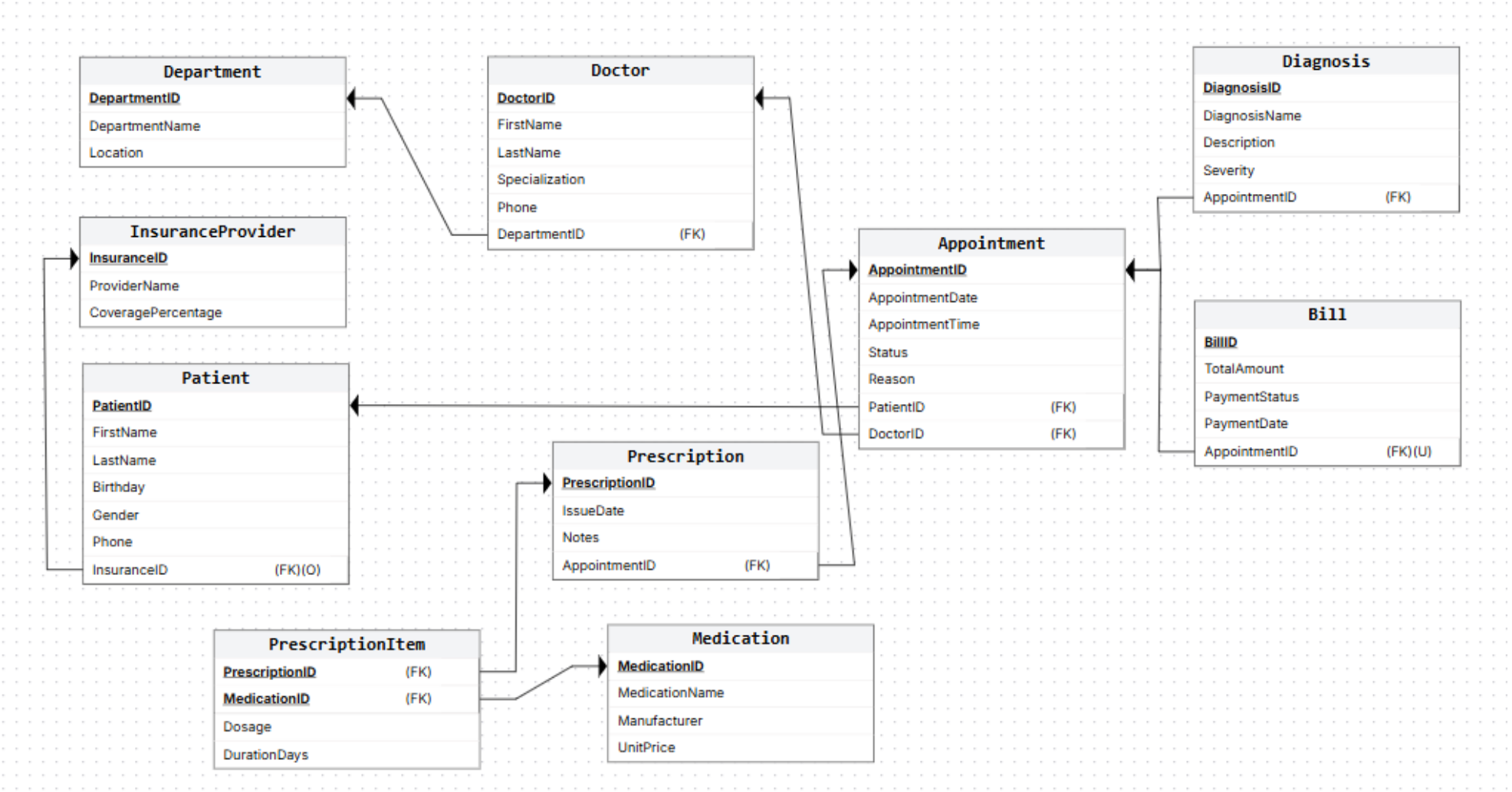
9 IsFor

Medication – PrescriptionItem

An item is for exactly one medication; a medication may appear in zero or many items.

Relational Schema

Each entity → a table. Each relationship → a foreign key or an intersection table.



From ERD to Relational Schema

Four conversion rules, each with a real example from our database.

1 Entity → Table

`Patient { FullName { FirstName, LastName } }` → **`Patient (FirstName, LastName)`**

Each ERD entity becomes a table. Composite attributes don't exist in the relational model, so FullName is dropped and its leaves (FirstName, LastName) become columns on their own.

2 One-to-Many → FK on the many side

`Department –< Doctor` → **`Doctor.DepartmentID (FK)`**

A doctor belongs to one department, but a department has many doctors. The FK goes on the many side. NOT NULL because every doctor must have a department.

3 One-to-One → FK with UNIQUE

`Appointment –|–| Bill` → **`Bill.AppointmentID (FK, UNIQUE)`**

Every bill belongs to exactly one appointment. The UNIQUE constraint prevents the same appointment from generating two bills. We put the FK on Bill because Bill is the mandatory side.

4 Many-to-Many → Intersection table

`Prescription >–< Medication` → **`PrescriptionItem (PK: PrescID + MedID)`**

Pure M:N can't be expressed directly. We create PrescriptionItem with two FKs forming a composite PK. Dosage and DurationDays live here because they describe the pairing, not either side.

CREATE TABLE — A Highlight

Our trickiest table: PrescriptionItem (the intersection table)

```
CREATE TABLE PrescriptionItem (
    PrescriptionID INTEGER NOT NULL,
    MedicationID INTEGER NOT NULL,
    Dosage VARCHAR NOT NULL,
    DurationDays INTEGER,
    PRIMARY KEY (PrescriptionID, MedicationID),
    FOREIGN KEY (PrescriptionID)
        REFERENCES Prescription(PrescriptionID),
    FOREIGN KEY (MedicationID)
        REFERENCES Medication(MedicationID)
);
```

Composite PK

Same medication can't appear twice in one prescription.

Two FKs

Link to both Prescription and Medication.

Dosage lives here

Not in Medication — different patients get different doses.

The Sample Dataset

Realistic, varied data designed to exercise every relationship.

5	Departments	**7**	Doctors	**10**	Patients
4	Insurance providers	**10**	Medications	**20**	Appointments
18	Diagnoses	**16**	Prescriptions	**25**	Prescription items

3 patients without insurance · 3 cancelled or scheduled appointments without bills · prescriptions with multiple medications

Query: Revenue Per Department

Touches 4 tables. Answers a real business question.

```
SELECT dep.DepartmentName,
       SUM(b.TotalAmount) AS TotalRevenue
FROM Department dep,Doctor doc, Appointment a, Bill b
WHERE dep.DepartmentID = doc.DepartmentID
AND doc.DoctorID = a.DoctorID
AND a.AppointmentID = b.AppointmentID
GROUP BY dep.DepartmentName
ORDER BY TotalRevenue DESC;
```

Result

Department	Revenue
Orthopedics	260
Cardiology	245
Pediatrics	180
Neurology	165
Dermatology	150

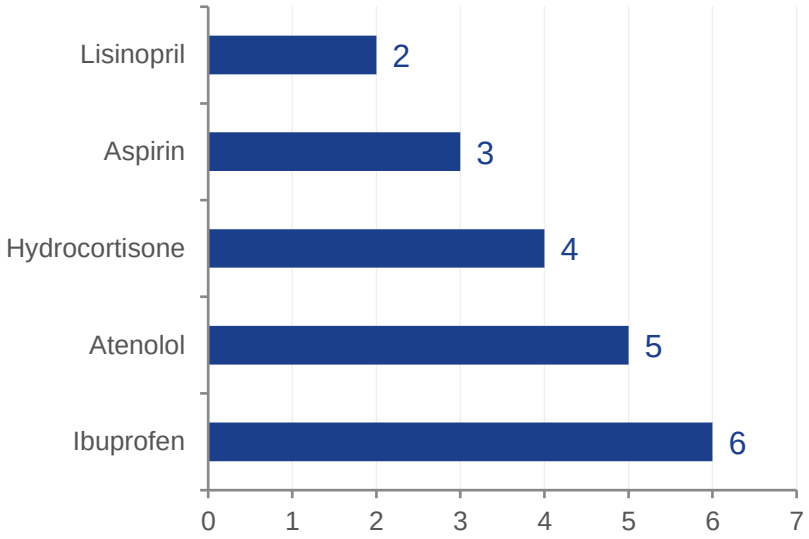
Why it's interesting

- Joining 4 Tables: Department, Doctor, Appointment, Bill all chained by FK
- SUM aggregate over groups, sorted to show top performer
- **This is exactly what the head of administration would ask**

Query: Most Prescribed Medications

Counting through the intersection table.

```
SELECT med.MedicationName,
       COUNT(pi.PrescriptionID) AS TimesPrescribed
FROM Medication med, PrescriptionItem pi
WHERE med.MedicationID = pi.MedicationID
GROUP BY med.MedicationID
ORDER BY TimesPrescribed DESC;
```



Why it's interesting

- Counts through the intersection table
- Useful for pharmacy ordering and stocking decisions
- Could uncover prescribing trends, over-prescribed antibiotics, seasonal patterns

Query: Patients Without Insurance

Short query, deep point: NULL is a first-class concept in our schema.

```
SELECT PatientID, FirstName, LastName, Phone
FROM Patient
WHERE InsuranceID IS NULL;
```

Result

ID	First	Last	Phone
3	Olivia	Williams	555-0103
6	Noah	Miller	555-0106
10	Ethan	Thomas	555-0110

Why it's interesting

- Only works because we modeled HasInsurance as optional so InsuranceID is nullable
- Many systems force every patient to have insurance, ours doesn't, and reality reflects that
- *Patients can receive medical help even without insurance*

Bonus: A Reusable View

CREATE VIEW — saves the query and lets us reuse it like a table.

```
-- Define it once...
CREATE VIEW TotalMedicationPrescribed AS
SELECT med.MedicationID,
       med.MedicationName,
       COUNT(pi.PrescriptionID) AS 'TimesPrescribed'
FROM Medication med, PrescriptionItem pi
WHERE med.MedicationID = pi.MedicationID
GROUP BY med.MedicationID;

-- ...and query it forever after, like a table:
SELECT * FROM TotalMedicationPrescribed;
```

Why it's useful

- Hides complexity, analysts can use the view without knowing it
- Acts like a read-only table, pluggable into other queries

All 25 Managerial Queries

SQL + Result for each — the full reference

Query 1

Display all patients along with their insurance provider name.

```
1 -- Query 1
2 -- Rationale: Display all patients along with their insurance provider name.
3 SELECT p.PatientID, p.FirstName, p.LastName, ip.ProviderName
4 FROM Patient p, InsuranceProvider ip
5 WHERE p.InsuranceID = ip.InsuranceID;
6
```

PatientID	FirstName	LastName	ProviderName
1	Emma	Johnson	HealthFirst
2	James	Smith	MediCare Plus
4	Liam	Brown	SafeGuard Insurance
5	Sophia	Davis	HealthFirst
7	Ava	Wilson	PrimeHealth
8	Lucas	Taylor	MediCare Plus
9	Mia	Anderson	SafeGuard Insurance

Query 2

Display the number of doctors in each department.

```
 8 -- Query 2
 9 -- Rationale: Display the number of doctors in each department.
10 SELECT dep.DepartmentName, COUNT(doc.DoctorID)
11 FROM Department dep, Doctor doc
12 WHERE dep.DepartmentID = doc.DepartmentID
13 GROUP BY dep.DepartmentName;
14
```

DepartmentName	COUNT(doc.DoctorID)
Cardiology	2
Dermatology	1
Neurology	1
Orthopedics	1
Pediatrics	2

Query 3

Display the total revenue generated per department.

```
15 -- Query 3
16 -- Rationale: Display the total revenue generated per department.
17 SELECT dep.DepartmentName, SUM(b.TotalAmount) AS TotalRevenue
18 FROM Department dep, Doctor doc, Appointment a, Bill b
19 WHERE dep.DepartmentID = doc.DepartmentID
20 AND doc.DoctorID = a.DoctorID
21 AND a.AppointmentID = b.AppointmentID
22 GROUP BY dep.DepartmentName
23 ORDER BY TotalRevenue DESC;
24
```

DepartmentName	TotalRevenue
Orthopedics	290
Cardiology	245
Pediatrics	180
Neurology	165
Dermatology	150

Query 4

Find which doctors handled the most appointments.

```
25
26 -- Query 4
27 -- Rationale: Find which doctors handled the most appointments.
28 SELECT doc.DoctorID, doc.FirstName, doc.LastName, COUNT(a.AppointmentID) AS TotalAppointments
29 FROM Doctor doc, Appointment a
30 WHERE doc.DoctorID = a.DoctorID
31 GROUP BY doc.DoctorID
32 ORDER BY TotalAppointments DESC;
33
```

DoctorID	FirstName	LastName	TotalAppointments
103	Michael	Walker	4
104	Emily	Hall	4
101	Robert	Clark	3
102	Sarah	Lewis	3
105	Daniel	Young	3
106	Jessica	King	2
107	David	Wright	1

Query 5

Display all unpaid bills along with the patient name and appointment date.

```
33
34 -- Query 5
35 -- Rationale: Display all unpaid bills along with the patient name and appointment date.
36 SELECT p.FirstName, p.LastName, a.AppointmentDate, b.TotalAmount, b.PaymentStatus
37 FROM Bill b, Appointment a , Patient p
38 WHERE b.AppointmentID = a.AppointmentID
39 AND a.PatientID = p.PatientID
40 AND b.PaymentStatus = 'Unpaid';
41
```

FirstName	LastName	AppointmentDate	TotalAmount	PaymentStatus
Olivia	Williams	2026-03-02	50	Unpaid
Mia	Anderson	2026-03-05	50	Unpaid
Emma	Johnson	2026-03-07	60	Unpaid
Noah	Miller	2026-03-12	65	Unpaid

Query 6

Display the most prescribed medications by total number of prescriptions.

```
42 -- Query 6
43 -- Rationale: Display the most prescribed medications by total number of prescriptions.
44 SELECT med.MedicationName, COUNT(pi.PrescriptionID) AS TimesPrescribed
45 FROM Medication med, PrescriptionItem pi
46 WHERE med.MedicationID = pi.MedicationID
47 GROUP BY med.MedicationID
48 ORDER BY TimesPrescribed DESC;
49
```

MedicationName	TimesPrescribed
Ibuprofen	6
Atenolol	4
Naproxen	3
Aspirin	3
Hydrocortisone Cream	3
Lisinopril	2
Cetirizine	2
Omeprazole	1
Amoxicillin	1

Query 7

Display each patient and the total number of appointments they have had.

```
50 -- Query 7
51 -- Rationale: Display each patient and the total number of appointments they have had.
52 SELECT p.PatientID, p.FirstName, p.LastName, COUNT(a.AppointmentID)
53 FROM Patient p, Appointment a
54 WHERE p.PatientID = a.PatientID
55 GROUP BY p.PatientID;
56
```

PatientID	FirstName	LastName	COUNT(a.AppointmentID)
1	Emma	Johnson	2
2	James	Smith	2
3	Olivia	Williams	2
4	Liam	Brown	2
5	Sophia	Davis	2
6	Noah	Miller	2
7	Ava	Wilson	2
8	Lucas	Taylor	2
9	Mia	Anderson	2
10	Ethan	Thomas	2

Query 8

Display all appointments for a specific date along with the doctor and patient names.

```
57 -- Query 8
58 -- Rationale: Display all appointments for a specific date along with the doctor and patient names.
59 SELECT a.AppointmentID, a.AppointmentTime, doc.FirstName, doc.LastName, p.FirstName, p.LastName
60 FROM Appointment a, Doctor doc, Patient p
61 WHERE a.DoctorID = doc.DoctorID
62 AND a.PatientID = p.PatientID
63 AND a.AppointmentDate = '2026-03-04';
64
```

AppointmentID	AppointmentTime	FirstName	LastName	FirstName	LastName
1006	08:30	Robert	Clark	Sophia	Davis
1007	10:00	Sarah	Lewis	Lucas	Taylor

Query 9

Display the average bill amount per department.

```sql
-- Query 9
-- Rationale: Display the average bill amount per department.
SELECT dep.DepartmentName, AVG(b.TotalAmount) AS AvgBill
FROM Department dep, Doctor doc, Appointment a, Bill b
WHERE dep.DepartmentID = doc.DepartmentID
AND doc.DoctorID = a.DoctorID
AND a.AppointmentID = b.AppointmentID
GROUP BY dep.DepartmentName;
```

DepartmentName	AvgBill
Cardiology	61.25
Dermatology	50
Neurology	55
Orthopedics	96.66666666666667
Pediatrics	45

Query 10

Find patients who have had more than 2 appointments.

```sql
-- Query 10
-- Rationale: Find patients who have had more than 1 appointment.
SELECT p.PatientID, p.FirstName, p.LastName, COUNT(a.AppointmentID) AS TotalAppointments
FROM Patient p, Appointment a
WHERE p.PatientID = a.PatientID
GROUP BY p.PatientID
HAVING TotalAppointments > 1;
```

PatientID	FirstName	LastName	TotalAppointments
1	Emma	Johnson	2
2	James	Smith	2
3	Olivia	Williams	2
4	Liam	Brown	2
5	Sophia	Davis	2
6	Noah	Miller	2
7	Ava	Wilson	2
8	Lucas	Taylor	2
9	Mia	Anderson	2
10	Ethan	Thomas	2

Query 11

Display all cancelled appointments along with the patient and doctor names.

```
82 -- Query 11
83 -- Rationale: Display all cancelled appointments along with the patient and doctor names.
84 SELECT a.AppointmentID, a.AppointmentDate, p.FirstName, p.LastName, doc.FirstName, doc.LastName
85 FROM Appointment a, Patient p , Doctor doc
86 WHERE a.PatientID = p.PatientID
87 AND a.DoctorID = doc.DoctorID
88 AND a.Status = 'Cancelled';
89
```

AppointmentID	AppointmentDate	FirstName	LastName	FirstName	LastName
1015	2026-03-09	Liam	Brown	Emily	Hall

Query 12

Display the total amount each patient has spent on bills.

```
89
90 -- Query 12
91 -- Rationale: Display the total amount each patient has spent on bills.
92 SELECT p.PatientID, p.FirstName, p.LastName, SUM(b.TotalAmount) AS TotalSpent
93 FROM Patient p, Appointment a, Bill b
94 WHERE p.PatientID = a.PatientID
95 AND a.AppointmentID = b.AppointmentID
96 GROUP BY p.PatientID
97 ORDER BY TotalSpent DESC;
98
```

PatientID	FirstName	LastName	TotalSpent
6	Noah	Miller	145
8	Lucas	Taylor	135
4	Liam	Brown	120
1	Emma	Johnson	120
5	Sophia	Davis	110
10	Ethan	Thomas	105
9	Mia	Anderson	105
7	Ava	Wilson	85
2	James	Smith	55
3	Olivia	Williams	50

Query 13

Display all diagnoses with severity marked as Severe.

```
98
99 -- Query 13
100 -- Rationale: Display all diagnoses with severity marked as Severe.
101 SELECT diag.DiagnosisName, diag.Description, a.AppointmentDate, p.FirstName, p.LastName
102 FROM Diagnosis diag, Appointment a, Patient p
103 WHERE diag.AppointmentID = a.AppointmentID
104 AND a.PatientID = p.PatientID
105 AND diag.Severity = 'Severe';
106
```

DiagnosisName	Description	AppointmentDate	FirstName	LastName
Arrhythmia	Irregular heartbeat observed	2026-03-01	Emma	Johnson
Torn Meniscus	Knee cartilage tear confirmed	2026-03-03	Liam	Brown
Migraine	Recurring migraine with aura	2026-03-11	Mia	Anderson
Atrial Fibrillation	Irregular rapid heart rate	2026-03-12	Noah	Miller

Query 14

Display each medication and the total revenue it has generated.

```
107 -- Query 14
108 -- Rationale: Display each medication and the total revenue it has generated.
109 SELECT med.MedicationName, med.UnitPrice, COUNT(pi.PrescriptionID),
110        med.UnitPrice * COUNT(pi.PrescriptionID) AS TotalRevenue
111 FROM Medication med, PrescriptionItem pi
112 WHERE med.MedicationID = pi.MedicationID
113 GROUP BY med.MedicationID
114 ORDER BY TotalRevenue DESC;
115
```

MedicationName	UnitPrice	COUNT(pi.PrescriptionID)	TotalRevenue
Atenolol	12	4	48
Hydrocortisone Cream	15	3	45
Ibuprofen	5	6	30
Lisinopril	11	2	22
Naproxen	6	3	18
Cetirizine	7	2	14
Aspirin	3	3	9
Amoxicillin	9	1	9
Omeprazole	8	1	8

Query 15

Display all patients who do not have insurance.

```
116 -- Query 15
117 -- Rationale: Display all patients who do not have insurance.
118 SELECT PatientID, FirstName, LastName, Phone
119 FROM Patient
120 WHERE InsuranceID IS NULL;
121
```

PatientID	FirstName	LastName	Phone
3	Olivia	Williams	555-0103
6	Noah	Miller	555-0106
10	Ethan	Thomas	555-0110

Query 16

Display the number of appointments per status (Scheduled, Completed, Cancelled).

```
122 -- Query 16
123 -- Rationale: Display the number of appointments per status (Scheduled, Completed, Cancelled).
124 SELECT Status, COUNT(AppointmentID)
125 FROM Appointment
126 GROUP BY Status;
127
```

Status	COUNT(AppointmentID)
Cancelled	1
Completed	17
Scheduled	2

Query 17

Display each doctor along with the total revenue they have generated from bills.

```sql
-- Query 17
-- Rationale: Display each doctor along with the total revenue they have generated from bills.
SELECT doc.DoctorID, doc.FirstName, doc.LastName, SUM(b.TotalAmount) AS TotalRevenue
FROM Doctor doc, Appointment a, Bill b
WHERE doc.DoctorID = a.DoctorID
AND a.AppointmentID = b.AppointmentID
GROUP BY doc.DoctorID
ORDER BY TotalRevenue DESC;

```

DoctorID	FirstName	LastName	TotalRevenue
104	Emily	Hall	290
105	Daniel	Young	165
103	Michael	Walker	150
102	Sarah	Lewis	140
106	Jessica	King	125
101	Robert	Clark	120
107	David	Wright	40

Query 18

Display all prescriptions that include more than one medication.

```sql

-- Query 18
-- Rationale: Display all prescriptions that include more than one medication.
SELECT pres.PrescriptionID, pres.IssueDate, COUNT(pi.MedicationID) AS MedicationCount
FROM Prescription pres, PrescriptionItem pi
WHERE pres.PrescriptionID = pi.PrescriptionID
GROUP BY pres.PrescriptionID
HAVING MedicationCount > 1;

```

PrescriptionID	IssueDate	MedicationCount
3001	2026-03-01	3
3002	2026-03-01	2
3004	2026-03-03	2
3006	2026-03-04	2
3009	2026-03-05	2
3011	2026-03-07	2
3013	2026-03-10	2
3015	2026-03-12	2

Query 19

Display the total number of diagnoses per department.

```sql
-- Query 19
-- Rationale: Display the total number of diagnoses per department.
SELECT dep.DepartmentName, COUNT(diag.DiagnosisID) AS TotalDiagnoses
FROM Department dep, Doctor doc, Appointment a, Diagnosis diag
WHERE dep.DepartmentID = doc.DepartmentID
AND doc.DoctorID = a.DoctorID
AND a.AppointmentID = diag.AppointmentID
GROUP BY dep.DepartmentName
ORDER BY TotalDiagnoses DESC;

```

DepartmentName	TotalDiagnoses
Cardiology	5
Pediatrics	4
Orthopedics	3
Neurology	3
Dermatology	3

Query 20

Display the insurance providers and the total bill amount covered by each.

```sql
-- Query 20
-- Rationale: Display the insurance providers and the total bill amount covered by each.
SELECT ip.ProviderName, ip.CoveragePercentage, SUM(b.TotalAmount) AS TotalBilled,
       SUM(b.TotalAmount * ip.CoveragePercentage / 100) AS AmountCovered
FROM InsuranceProvider ip, Patient p, Appointment a, Bill b
WHERE ip.InsuranceID = p.InsuranceID
AND p.PatientID = a.PatientID
AND a.AppointmentID = b.AppointmentID
GROUP BY ip.InsuranceID;

```

ProviderName	CoveragePercentage	TotalBilled	AmountCovered
HealthFirst	80	230	184
MediCare Plus	70	190	132
SafeGuard Insurance	90	225	202
PrimeHealth	60	85	51

Query 21

Display the number of appointments per status for each department.

```
164
165 -- Query 21
166 -- Rationale: Display the number of appointments per status for each department.
167 SELECT dep.DepartmentName, a.Status, COUNT(a.AppointmentID)
168 FROM Department dep, Doctor doc, Appointment a
169 WHERE dep.DepartmentID = doc.DepartmentID
170 AND doc.DoctorID = a.DoctorID
171 GROUP BY dep.DepartmentName, a.Status;
172
```

DepartmentName	Status	COUNT(a.AppointmentID)
Cardiology	Completed	4
Cardiology	Scheduled	1
Dermatology	Completed	3
Dermatology	Scheduled	1
Neurology	Completed	3
Orthopedics	Cancelled	1
Orthopedics	Completed	3
Pediatrics	Completed	4

Query 22

Display the busiest days by number of appointments.

```
172
173 -- Query 22
174 -- Rationale: Display the busiest days by number of appointments.
175 SELECT AppointmentDate, COUNT(AppointmentID) AS TotalAppointments
176 FROM Appointment
177 GROUP BY AppointmentDate
178 ORDER BY TotalAppointments DESC;
179
```

AppointmentDate	TotalAppointments
2026-03-01	2
2026-03-03	2
2026-03-04	2
2026-03-05	2
2026-03-07	2
2026-03-08	2
2026-03-10	2
2026-03-12	2
2026-03-02	1
2026-03-06	1
2026-03-09	1
2026-03-11	1

Query 23

Display each patient along with their most recent appointment date.

```
179
180 -- Query 23
181 -- Rationale: Display each patient along with their most recent appointment date.
182 SELECT p.PatientID, p.FirstName, p.LastName, MAX(a.AppointmentDate) AS LastVisit
183 FROM Patient p, Appointment a
184 WHERE p.PatientID = a.PatientID
185 GROUP BY p.PatientID;
186
```

PatientID	FirstName	LastName	LastVisit
1	Emma	Johnson	2026-03-07
2	James	Smith	2026-03-08
3	Olivia	Williams	2026-03-08
4	Liam	Brown	2026-03-09
5	Sophia	Davis	2026-03-10
6	Noah	Miller	2026-03-12
7	Ava	Wilson	2026-03-07
8	Lucas	Taylor	2026-03-10
9	Mia	Anderson	2026-03-11
10	Ethan	Thomas	2026-03-12

Query 24

Display the total number of patients each insurance provider covers.

```
186
187 -- Query 24
188 -- Rationale: Display the total number of patients each insurance provider covers.
189 SELECT ip.ProviderName, COUNT(p.PatientID)
190 FROM InsuranceProvider ip, Patient p
191 WHERE ip.InsuranceID = p.InsuranceID
192 GROUP BY ip.ProviderName;
193
```

ProviderName	COUNT(p.PatientID)
HealthFirst	2
MediCare Plus	2
PrimeHealth	1
SafeGuard Insurance	2

Query 25

Create a view that shows total quantity of each medication prescribed.

```
193
194 -- Query 25
195 -- Rationale: Create a view that shows total quantity of each medication prescribed.
196 CREATE VIEW TotalMedicationPrescribed AS
197 SELECT med.MedicationID, med.MedicationName, COUNT(pi.PrescriptionID) AS 'TimesPrescribed'
198 FROM Medication med, PrescriptionItem pi
199 WHERE med.MedicationID = pi.MedicationID
200 GROUP BY med.MedicationID;
201
202 SELECT * FROM TotalMedicationPrescribed;
```

MedicationID	MedicationName	TimesPrescribed
401	Amoxicillin	1
402	Ibuprofen	6
403	Atenolol	4
404	Cetirizine	2
406	Omeprazole	1
407	Hydrocortisone Cream	3
408	Aspirin	3
409	Lisinopril	2
410	Naproxen	3

That's our database.

Correct by design. Fast by design.

10

tables, in 3NF

5

indexes where they matter

25

real-world queries



Questions?

Thank you